Native PendleNativeLPSY Audit Report

Oct 23, 2025

Table of Contents

Summary

This report has been prepared for PendleNativeLPSY smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**PendleNativeLPSY**
Codebase	**https://github.com/Native-org/Pendle-SY**
Commit	**eed2e81042a1b292fbbdd7badcad5806de1f17d3**
Language	**Solidity**

Audit Summary

Delivery Date	**Oct 23, 2025**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**3**

[WP-L1] By leveraging SY to arbitrarily specify the redeem receiver, any WNLP holder's address can be DoS'ed, effectively blocking transfers temporarily.

Low

Issue Description

WNLP doesn't allow transfers if the sender is within the deposit cooldown.

Since SY can specify the receiver for `redeem()` , an attacker can trigger a redeem to any target address and effectively prevent that address from sending WNLP.

https://github.com/Native-org/Pendle-SY/blob/main/contracts/core/StandardizedYield/implementations/Native/PendleNativeLPSY.sol#L43-L53

```
    function _redeem(
        address receiver,
        address tokenOut,
        uint256 amountSharesToRedeem
    ) internal override returns (uint256 amountTokenOut) {
        // update last deposit timestamp in WNLP
        IWNLP(WNLP).updateLastDepositTimestamp(receiver, block.timestamp);

        amountTokenOut = amountSharesToRedeem;
        _transferOut(tokenOut, receiver, amountTokenOut);
    }
```

```
function redeem(
        address receiver,
        uint256 amountSharesToRedeem,
        address tokenOut,
        uint256 minTokenOut,
        bool burnFromInternalBalance
    ) external nonReentrant returns (uint256 amountTokenOut) {
        if (!isValidTokenOut(tokenOut)) revert Errors.SYInvalidTokenOut(tokenOut);
        if (amountSharesToRedeem == 0) revert Errors.SYZeroRedeem();

```

```
        if (burnFromInternalBalance) {
            _burn(address(this), amountSharesToRedeem);
        } else {
            _burn(msg.sender, amountSharesToRedeem);
        }

        amountTokenOut = _redeem(receiver, tokenOut, amountSharesToRedeem);
        if (amountTokenOut < minTokenOut) revert Errors.SYInsufficientTokenOut(amountTokenOut, minTokenOut);
        emit Redeem(msg.sender, receiver, tokenOut, amountSharesToRedeem, amountTokenOut);
    }

```

https://github.com/Native-org/v2-core/blob/feat/wrap-nlp/src/WNLP.sol#L213

```
/// @notice Update user's last deposit timestamp
    /// @dev This is a emergency function, should only be called when certain protocols
    ///      need to bypass cooldown restrictions in exceptional circumstances.
    /// @param user User address to update
    /// @param timestamp New timestamp for the user
    function updateLastDepositTimestamp(address user, uint256 timestamp) external {
        require(trustedOperators[msg.sender], ErrorsLib.NotTrustedOperator());
        require(timestamp <= block.timestamp, ErrorsLib.InvalidAmount());

        lastDepositTimestamp[user] = timestamp;

        emit DepositTimestampUpdated(user, timestamp);
    }
```

https://github.com/Native-org/v2-core/blob/feat/wrap-nlp/src/WNLP.sol#L332-L345

```
function _transfer(address from, address to, uint256 amount) internal override {
    require(to != address(this), ErrorsLib.TransferToContract());

    // Unlike NLP's transfer validation, wNLP allows transfers if either the sender
    // or the recipient is in the redeemCooldownExempt, providing more flexibility
```

```
    require(
        lastDepositTimestamp[from] + nlp.minRedeemInterval() <= block.timestamp || nlp.redeemCooldownExempt(from)
            || nlp.redeemCooldownExempt(to),
        ErrorsLib.TransferInCooldown()
    );

    super._transfer(from, to, amount);
}
```

Status

✓ Fixed

[WP-I2] Consider using `SYBaseV2`

Informational

Issue Description

The current implementation inherits from `SYBase` .

Pendle has added new functions in `SYBaseV2` and removed the permit functionality.

https://github.com/Native-org/Pendle-SY/blob/b50728d55dcbc6db209c7c9d92449917f77583e7/contracts/core/StandardizedYield/implementations/Native/PendleNativeLPSY.sol

```
@@ 1,2 @@

import "../../SYBase.sol";
import "../../../../interfaces/Native/INLP.sol";
import "../../../../interfaces/Native/IWNLP.sol";

contract PendleNativeLPSY is SYBase {
@@ 9,99 @@
}
```

Status

✓ Fixed

[WP-N3] Unused library `PMath`

Issue Description

The contract does not use any functions from `PMath` .

https://github.com/Native-org/Pendle-SY/blob/main/contracts/core/StandardizedYield/implementations/Native/PendleNativeLPSY.sol#L9

```
using PMath for uint256;
```

Status

✓ Fixed

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.